AI-Assisted Vulnerability Discovery for IoT and Embedded Linux: Statement of Work Proposal

Abstract

This project investigates how increasing levels of AI involvement change vulnerability discovery for IoT and embedded-Linux software. We organize the work into three stages. Stage 1 establishes a reproducible human-led baseline using traditional security tools on controlled C/C++ software with known vulnerable behavior. Stage 2 studies published AI-assisted techniques and evaluates selected human-guided uses of LLMs within that baseline workflow. Stage 3 develops a student-scale, Naptime-inspired agent that can inspect source code, invoke analysis and runtime tools, and validate vulnerability hypotheses using observable program behavior. Across the stages, we emphasize controlled tasks, deterministic runtime evidence, reproducible environments, and comparable measurements of validation success, false candidates, time, manual intervention, tool use, cost, and failure modes. The goal is not zero-day discovery, but a practical evaluation of where AI assistance and agentic autonomy improve, complicate, or fail to improve a traditional vulnerability-research workflow.

Introduction and Motivation

Internet of Things (IoT) and embedded-Linux devices are widely deployed, and a vulnerability in their firmware or network-facing services can have serious security consequences. Finding such vulnerabilities is difficult. Firmware may depend on specific hardware, use undocumented protocols, expose hidden interfaces, or contain multiple interacting components. Traditional vulnerability discovery therefore relies on skilled analysts combining techniques such as fuzzing, attack-surface discovery, static and taint analysis, and dynamic analysis. This approach is effective, but it is also slow and largely manual.

Recent work applies large language models (LLMs) to many steps of this workflow, including reading code, inferring taint sources and sinks, drafting fuzzing seeds and harnesses, and explaining crashes. Agentic systems such as Google's Project Naptime and Big Sleep suggest that an LLM given suitable tools can carry out an iterative, hypothesis-driven research loop that ends in verifiable evidence. However, these results are not easy to compare directly: they use different targets, give the model or analyst different amounts of information, and judge success by different criteria.

Problem Statement

It is therefore unclear where AI assistance genuinely improves a traditional workflow, where it adds cost or unreliability, and how much autonomy an agent can usefully be given. Our problem is to compare three levels of AI involvement on the same task: a traditional human-led workflow, a human-guided AI-assisted workflow, and an agentic workflow in which an LLM selects and invokes tools on its own. The comparison is meaningful only if the task is held fixed, so all three conditions share one target, one information boundary, one environment, and one validation oracle.

Research question: under a shared target, information boundary, environment, and validation oracle, how do traditional, AI-assisted, and agentic analysis differ in validated success, false candidate findings, reproducibility, human intervention, cost, and failure modes?

Terminology is used consistently throughout the document: a candidate finding is a suspected vulnerability that has not yet passed validation; a validated finding is a candidate that satisfies the shared deterministic oracle; and the oracle is the predefined runtime rule used to decide whether the claimed behavior is actually reproduced.

Approach and Document Overview

This project addresses that question in three stages that share a controlled C/C++ target with known ground truth: a human-led traditional baseline (Stage 1), human-guided AI assistance applied to that baseline (Stage 2), and a reduced, Naptime-inspired agent that invokes tools and validates its own hypotheses (Stage 3). Findings count only when they pass the shared oracle, and the goal is not to discover new zero-day vulnerabilities but to rediscover and validate

known vulnerable behavior in a controlled setting and learn where AI helps, where it does not, and why. The remainder of this document presents the project goals and the overall project plan, followed by the Statements of Work for Stage 1, Stage 2, and Stage 3 in order, each covering related work, experimental design, and project planning. It closes with the shared experimental limitations and a conclusion covering the whole project. Section numbering within each stage follows that stage's own Statement of Work.

Project Goals

The project has five goals. For each, the stage-level goals in the Statements of Work below refine them.

- **Establish a reproducible traditional baseline (Stage 1).** Select and fix one primary target, a baseline protocol, and a deterministic oracle, and complete at least one vulnerability investigation validated by runtime evidence such as a reproducible crash or sanitizer report.
- **Evaluate AI-assisted techniques (Stage 2).** Survey published AI-assisted techniques by pipeline step, reproduce selected results where public artifacts allow, and apply one or two use cases (such as static-analysis triage, fuzzing seed and harness generation, or crash interpretation) to the baseline workflow. Success is a recorded comparison of AI verdict accuracy, harness build success, cost, and failure modes against the Stage 1 baseline.
- **Build a reduced agentic workflow (Stage 3).** Implement a student-scale, Naptime-inspired agent with source browsing, Python execution, target execution, and a controller that verifies claimed findings. Success is one end-to-end investigation completed and verified by deterministic runtime evidence.
- **Run a controlled comparison across the three stages.** Using the shared target, information boundary, environment, and oracle, compare source-only and runtime-verified agent behavior and study how a fixed harness interacts with at least two model capability tiers, with repeated trials to expose run-to-run variation.
- **Document results, limitations, and reusable artifacts.** Record trajectories, metrics, cost, and failure modes, state the shared experimental limitations explicitly, and hand each stage's environments, scripts, prompts, and logs to the next so that the work can be reproduced.

Overall Project Plan

The Statements of Work below give the detailed plan for each stage. This section brings together the team responsibilities, the full project schedule, the budget, and the project-level risks that span all three stages.

Team Responsibilities

The team has three members, and each leads one stage: Jingqi leads Stage 1, Albert leads Stage 2, and Sherry leads Stage 3. The lead plans the stage, keeps it on schedule, and makes sure its deliverables are completed and handed on, but the work itself is shared. All three members contribute to every stage, for example by building environments, running experiments, reviewing results, and writing up findings. Because the stages share the same target, information boundary, environment, and oracle, this keeps them consistent with one another and means no stage depends on a single person.

Team member	Stage led	Lead responsibilities
Jingqi	Stage 1 — Traditional / existing vulnerability discovery	Coordinates target selection, the information boundary and oracle, the reproducible environment, the baseline investigation, baseline measurements, and the handoff of the protocol and artifacts to Stage 2.
Albert	Stage 2 — Case studies of AI-assisted vulnerability discovery	Coordinates the survey of published techniques, the reproductions, the application of chosen use cases to the Stage 1 workflow, the recording of model, prompt, cost, and failure modes, and the shortlist of mechanisms for Stage 3.
Sherry	Stage 3 — Naptime-inspired agentic vulnerability discovery	Coordinates the reduced agent architecture, tools, controller, and logging, the source-only and runtime-verified experiments, the model–harness experiment, and the analysis of metrics

Team member	Stage led	Lead responsibilities
		and failure modes.

Across all stages, every member takes on tasks in other stages as the schedule requires, reviews each stage's protocol, results, and limitations, contributes to the progress updates, the final presentation, and the final report, and keeps prompts, scripts, and logs in shared version control.

Project Schedule

The schedule below combines the three stage timelines and the course milestones. The Stage 2 literature study can begin before Stage 1 finishes, and Stage 3 begins from the Stage 2 shortlist.

Dates	Milestone	Lead	Main output
Oct. 3 – Oct. 20	Stage 1 — Traditional baseline	Jingqi	Finalized target, protocol, and oracle; reproducible environment; validated baseline investigation; baseline measurements
Between Stage 1 and Stage 2	Progress Update #1	Team	Selected target, traditional workflow, initial results, validation evidence, and candidate steps for AI assistance
Oct. 21 – Nov. 3	Stage 2 — AI-assisted case studies	Albert	Technique summaries, reproduction notes, initial AI-assisted results against the baseline, failure modes, and a shortlist for Stage 3
Nov. 4	Progress Update #2	Team	Summary of AI-assisted approaches, initial comparison with the baseline, and selected techniques for Stage 3
Nov. 4 – Nov. 17	Stage 3A — Architecture and MVP	Sherry	Reduced architecture, agent loop, source-browser and runtime tools, one end-to-end run on a simple target
Nov. 18	Progress Update #3	Team	Reference architecture, reduced design, MVP demonstration, blockers, and Stage 3B plan
Nov. 18 – Dec. 3	Stage 3B — Integration, experiments, and evaluation	Sherry	Controlled experiments, model–harness experiment, metrics, repeated trials, and failure-mode analysis
Dec. 4	Final presentation	Team	Stage 1 → Stage 2 → Stage 3 progression, agent demo, results, limitations, and future work
Dec. 4 – Dec. 9	Final report	Team	Consolidated experiments, final analysis, methodology, limitations, and lessons learned

Leads coordinate each stage, and all members contribute to the work.

Project Budget

The project is software-focused, so the expected budget is small. Most resources are open source or already available through the team and the course, and the main potential cost is LLM API usage. API cost is also recorded as an experimental metric.

Item	Source	Expected cost
Build tools, sanitizers, debugger, Git, and open-source analysis and fuzzing tools	Open source	None
Compute (Linux or Docker environment)	Existing team laptops and school compute	None

Item	Source	Expected cost
Embedded hardware (optional extension only)	Course-provided	None
LLM API usage, Stage 2 (small-prompt case studies and reproductions)	Paid API access	$5
LLM API usage, Stage 3 (agent experiments and model–harness comparison)	Paid API access	$10
Total project cap		**$15**

We expect total API spend of about $15, with a hard cap of $50. Most work will use free, low-cost, and inexpensive model tiers and free credits where available, and we will keep prompts small, giving the model only the context it needs.

Project-Level Risks and Backup Plans

Each Statement of Work lists the risks specific to its stage. The following risks span the whole project and come from the shared evaluation design.

Risk	Backup plan
Known targets or CVEs appeared in model training data and overstate AI performance	State the limitation explicitly, use additional targets as generalization checks where feasible, and describe results as controlled-case evidence rather than general capability
The information given to the analyst, the model, or the agent differs between conditions and changes task difficulty	Freeze and log the information boundary for each comparison, and record any deviation as a limitation rather than silently reconciling it
Environment and setup time dominates the investigation	Record setup time and investigation time separately, prioritize one validated investigation, and fall back to the backup target if the primary target is too costly to set up
The team already knows the Stage 1 solution when Stage 2 reruns the same task	Account for the carryover when interpreting results, use a second historical vulnerability if available, and measure selected AI artifacts (such as seed quality, harness build success, and triage accuracy) where a clean end-to-end rerun is not possible
A late or incomplete handoff from one stage delays the next	Let Stage 2 begin with literature study and public-target reproductions, and let Stage 3A begin from the public Naptime and Baby Naptime materials, so neither stage is blocked while waiting for the previous one
A team member becomes overloaded or blocked	Because the work is shared, any member can pick up tasks. Keep prompts, scripts, logs, and protocol documents in shared version control so work can move between members
Primary target is too large or difficult to build or reproduce	Scope down to one component, parser, or historical vulnerability, or switch to the smaller controlled backup target
LLM output is unreliable and varies between runs	Fix the model, version, and settings where possible, repeat important runs, log complete trajectories, and report variance and failures rather than relying on a single run

Stage 1 — Traditional / Existing Vulnerability Discovery

1. Research / Related Work

1.1 Overview of Existing Vulnerability-Discovery Directions

IoT vulnerability discovery involves several complementary techniques rather than a single testing method. Some approaches focus on identifying attack surfaces, some analyze potentially vulnerable data flows, while others execute or actively test firmware and devices. IoT environments introduce additional challenges because firmware may depend on specific hardware, use undocumented protocols, expose hidden interfaces, or contain multiple interacting components.

For Stage 1, we focus on established non-AI vulnerability-discovery techniques and use them to identify a reproducible traditional baseline for later stages of the project.

Direction	Main Purpose	Typical Traditional Techniques	Main Value	Main Challenge
Fuzzing	Automatically discover failures by exercising inputs	Greybox fuzzing, protocol fuzzing, crash monitoring	Mature tools and measurable results	Requires suitable input interface and execution environment
Interface / Attack-Surface Discovery	Identify exposed or hidden entry points	Route analysis, firmware path extraction, active probing	Particularly relevant to IoT web interfaces	Hidden routes and parameters may be difficult to reconstruct
Black-Box Device Testing	Test devices without relying on source code or firmware internals	Traffic analysis, request mutation, response-guided testing	Applicable to real commercial devices	Ground truth and internal coverage are difficult to measure
Static / Taint Analysis	Identify suspicious data flows without execution	Source/sink identification, binary analysis, taint propagation	Can narrow a large firmware search space	False positives and incomplete program models
Firmware Rehosting / Dynamic Analysis	Execute firmware in a controlled environment	QEMU/system emulation, environment repair	Enables debugging and repeatable dynamic testing	Firmware often depends on missing hardware/environment state
Cross-Binary / Cross-Language Analysis	Track vulnerability paths across firmware components	IPC/file-flow reconstruction, cross-binary taint	Reflects realistic embedded-Linux architecture	Significantly more complex than single-program analysis

1.2 Fuzzing

Fuzzing is one of the most established techniques for automatically discovering software vulnerabilities. A fuzzer repeatedly generates or mutates test inputs and executes the target while monitoring crashes, memory errors, unexpected responses, or newly reached execution paths. In IoT systems, fuzzing can be more difficult because firmware may run on a different architecture, rely on hardware-specific components, or communicate through undocumented protocols.
IoTFuzzer [1] demonstrates an app-assisted approach in which the official mobile application is used to generate and mutate messages sent to real IoT devices. This allows testing without directly instrumenting the firmware.
FIRM-AFL [2] instead targets firmware that can be emulated and combines system-mode and user-mode emulation to support higher-throughput greybox fuzzing.
These works show two useful traditional baselines: external device fuzzing and emulated firmware fuzzing.

1.3 Interface / Attack-Surface Discovery

Before vulnerabilities can be tested, the analyst must determine where attacker-controlled input can enter the system. IoT devices often contain hidden or undocumented web endpoints, administrative routes, diagnostic functions, or backend APIs that are not exposed through the normal frontend.
Game of Hide-and-Seek / IoTScope [3] analyzes embedded web applications to identify interfaces implemented in firmware but hidden from the normal user interface. It combines firmware analysis with active probing of real devices. This direction is especially relevant when the selected target contains a web-management interface or other firmware-defined network endpoints.

1.4 Black-Box Device Testing

Black-box testing analyzes a device from the outside without requiring source code or knowledge of the internal firmware implementation. The tester interacts with network-visible interfaces, modifies requests, and uses observed responses to guide further testing.
This approach is useful for commercial IoT devices where firmware may be encrypted, difficult to extract, or difficult to emulate.
Snipuzz [4] uses response-guided black-box fuzzing to infer meaningful message regions and selectively mutate them without relying on firmware instrumentation.
Black-box testing therefore provides a practical approach for real-device experiments, although measuring complete coverage or ground truth can be difficult.

1.5 Static / Taint Analysis

Static analysis examines source code or binaries without executing them. Taint analysis tracks attacker-controlled input from a source, such as a network request, toward a security-sensitive sink, such as command execution or unsafe memory operations.
For IoT firmware, static analysis can reduce a large search space by identifying suspicious paths before dynamic validation.
Operation Mango [5] focuses on scalable discovery of taint-style vulnerabilities in binary firmware services. It demonstrates how traditional program analysis can be applied to large collections of firmware binaries.
A limitation of this direction is that a suspicious static path does not necessarily represent a real exploitable vulnerability, so findings generally require later validation.

1.6 Firmware Rehosting / Dynamic Analysis

Dynamic vulnerability analysis often requires firmware to run in a controlled environment. IoT firmware may depend on a particular processor architecture, filesystem, NVRAM state, network configuration, or hardware peripheral, making direct execution difficult.
Firmware rehosting attempts to reproduce enough of the device environment using emulation so that important firmware components can execute on an analyst-controlled machine.
FIRMADYNE [6] is a foundational system for automated dynamic analysis of Linux-based embedded firmware using full-system emulation.
FirmAE [7] extends this approach by addressing common inconsistencies between real firmware environments and emulated environments, improving the success rate of firmware execution.
Rehosting can enable later debugging, fuzzing, and vulnerability reproduction, but setup complexity makes it a higher-risk baseline for a short project stage.

1.7 Cross-Binary / Cross-Language Analysis

IoT firmware usually contains multiple interacting programs rather than one standalone executable. User-controlled input may enter through one process, be written to a file or IPC channel, and eventually reach a vulnerable operation inside another process.
Analysis limited to a single binary can therefore miss complete vulnerability paths.
KARONTE [8] studies insecure interactions between multiple firmware binaries and tracks data across those interactions. This demonstrates the importance of considering firmware as a connected software system rather than a collection of independent programs.
This direction is highly relevant to realistic embedded-Linux firmware but is also one of the more technically complex options for the project baseline.

1.8 Related-Work Takeaway

The reviewed techniques address different stages of a traditional IoT vulnerability-discovery workflow:
Firmware execution / rehosting → Attack-surface discovery → Static or cross-component analysis → Active testing / fuzzing → Finding validation
For Stage 1, the goal is not to reproduce every technique. Instead, the literature review will be used to select a narrow, reproducible workflow that can be executed within the Stage 1 schedule.
The baseline should preferably:

- operate on a reproducible target;
- provide observable and measurable outputs;
- support validation of findings;
- require a meaningful amount of analyst reasoning or setup;
- remain feasible within the available time and resources.

2. Experimental Design

2.1 Objective

The Stage 1 baseline is designed to support the later stages:

- Stage 1: human analyst + traditional tools;
- Stage 2: human-guided AI assistance applied to selected steps of the Stage 1 workflow;
- Stage 3: a reduced agentic workflow that can reuse compatible tools, targets, and validation methods while autonomously selecting actions and iteratively validating findings.

Stage 1 therefore delivers a baseline protocol, not only a baseline result. The primary comparison in Stages 2 and 3 should reuse the same target, information boundary, environment, and validation oracle wherever possible, while varying the degree of AI assistance or autonomy. Additional targets may be used for practice, development, or generalization checks.

2.2 Target Scope

Rather than testing several unrelated classes of IoT systems, Stage 1 will select one primary controlled software target that can also support the later AI-assisted and agentic stages.

The preferred primary target should be:

- source available;
- written primarily in C/C++;
- compilable and runnable on Linux or in a container;
- small enough to inspect within the project schedule;
- relevant to IoT or embedded-Linux systems;
- accessible through a clear external input, such as a file, request, protocol message, or network interface;
- compatible with runtime instrumentation such as AddressSanitizer or a debugger;
- associated with at least one documented historical vulnerability or other known ground truth.

A network-facing or protocol-processing component is preferred because it provides stronger IoT relevance and a clear attacker-controlled input.

Candidate targets are shortlisted as follows:

Primary shortlist: libcoap or libmodbus, each restricted to a fixed historical revision, one protocol-facing component, and one or two documented vulnerabilities.

Alternative targets: Eclipse Mosquitto if scoped to one packet-processing component; one BusyBox utility rather than BusyBox as a whole; IoTGoat or another intentionally vulnerable target as backup or practice.

Before Stage 1 experimentation begins, the team will freeze the selected repository, revision, component/input interface, historical vulnerability task, and backup target. These fixed values become part of the baseline protocol.

The baseline protocol will record:

- target repository and exact revision/version;
- selected component and attacker-controlled input/interface;
- historical vulnerability(s) used as ground truth;
- backup target and switching criterion.

2.3 Baseline Task Definition

The core Stage 1 task is to perform a traditional investigation of a known or historically vulnerable software target and determine whether the vulnerable behavior can be identified and validated using conventional analysis tools.

The exact vulnerability type will depend on the selected target. Preference will be given to vulnerabilities that provide deterministic validation evidence, such as:

- a reproducible crash;
- an AddressSanitizer report;
- a clearly incorrect security-sensitive program behavior;
- another repeatable runtime condition associated with the documented vulnerability.

The information boundary will be fixed before the experiment and reused across comparison conditions. It will explicitly state whether the analyst is told the CVE identifier, CWE type, component name, protocol/interface description, or only that a historical vulnerability exists. No condition will be given the final vulnerable function or line, a ready-made proof input/PoC, or the final solution. Ground truth remains available to the evaluator for scoring

and validation, not as investigation guidance.

The baseline protocol therefore consists of the fixed target and revision, information boundary, build/runtime environment, allowed traditional tools and versions, task definition, deterministic oracle, and required trajectory logging. The Stage 1 result (for example, whether a vulnerability was validated and how the investigation proceeded) is an observation produced under this protocol.

2.4 Traditional Baseline Workflow

The baseline will follow the same general vulnerability-research loop that later stages will progressively augment or automate.

Step 1 — Prepare and understand the target

- build the selected version;
- identify the relevant component and external input;
- document the execution environment;
- prepare the historical vulnerability information used as ground truth.

Step 2 — Inspect the attack surface and source

Use conventional tools to identify relevant entry points and source locations. Depending on the target, this may include:

- source-code browsing;
- text/reference search;
- manual control-flow or data-flow reasoning;
- static-analysis tools where useful.

Step 3 — Form candidate vulnerability hypotheses

Based on the available source and tool results, identify code paths or behaviors that may correspond to the target vulnerability.

Step 4 — Construct or obtain test input

Create an input capable of exercising the suspected code path.
Depending on the final target, this may involve:

- manual input construction;
- a small test harness;
- existing test utilities;
- fuzzing or mutation tools where appropriate.

Fuzzing is therefore treated as a possible tool within the baseline, rather than a required separate project direction.

Step 5 — Execute and observe

Run the target with the candidate input and collect observable evidence such as:

- program output;
- exit status;
- crash information;
- AddressSanitizer output;
- debugger observations.

Step 6 — Validate or reject the hypothesis

A vulnerability finding will be considered validated only when supported by reproducible program behavior or other deterministic evidence.
Unsupported or non-reproducible candidate findings will be recorded separately.

Step 7 — Document the investigation trajectory

Record:

- tools used;
- important analyst actions;
- candidate hypotheses;
- test inputs;
- Failed attempts and tools used during the investigation;
- runtime results;
- validated findings;
- limitations and manual bottlenecks.

This documentation will provide the baseline that Stages 2 and 3 can reuse.

2.5 Relationship to the Techniques Reviewed in Section 1

The techniques reviewed in Section 1 remain useful as possible components of the baseline, but Stage 1 will not attempt to reproduce all six areas.
For the core experiment:

- source/static analysis may support source inspection and candidate identification;
- dynamic analysis will provide runtime evidence;
- fuzzing may be used when automated input generation is useful;
- attack-surface discovery may be used to identify relevant external inputs.

More complex techniques such as full firmware rehosting, cross-binary analysis, and commercial-device black-box testing are not required for minimum Stage 1 success. They remain possible extensions if the selected target requires them and sufficient time is available.

2.6 Experimental Conditions and Measurements

Stage 1 is primarily intended to establish a reproducible baseline protocol and characterize the traditional workflow. Measurements are collected so later stages can compare against the same task without treating a single novice-analyst run as a universal human-performance benchmark.

Controlled protocol

The following will be fixed and documented before the measured run:

- target repository, exact revision/version, and selected component;
- historical vulnerability task and ground truth held by the evaluator;
- information boundary, including exactly what vulnerability/context information is supplied;
- build configuration and runtime environment;
- allowed traditional tools and tool versions;
- deterministic validation oracle;
- logging requirements for actions, hypotheses, failed attempts, inputs, and runtime evidence.

Core baseline metrics

- Validated finding: whether the historical vulnerable behavior is successfully identified and passes the oracle;
- False candidate findings: suspected vulnerabilities that fail validation;
- Reproducibility: whether the validated behavior can be reproduced reliably;
- Time: setup time and investigation time recorded separately, with time treated as descriptive;

- Investigation effort: major analyst steps/actions recorded as a workflow trace.

Supporting measurements

- Human interventions and manual reasoning required;
- Oracle evidence, such as ASan output, crash evidence, or another deterministic runtime condition;
- Failure or bottleneck category, such as setup, attack-surface identification, hypothesis formation, input construction, code-path reachability, tool failure, or environment failure;
- failed attempts;
- Any deviations from the frozen protocol.

With only one or two historical vulnerabilities, Stage 1 will report validated success, false candidate findings, reproducibility, and qualitative failure modes rather than statistically meaningful precision, recall, or F1. Human action counts are also not assumed to be equivalent to later agent tool-call counts.

A further confound is analyst carryover: the same team members may already know the Stage 1 solution. Where an end-to-end rerun would be strongly contaminated by that knowledge, Stage 2 will either use a second historical vulnerability under the same protocol or restrict the comparison to AI-produced artifacts at selected steps (for example harness build success, seed quality, or triage), rather than claiming a clean end-to-end human-vs-AI comparison.

3. Project Planning

3.1 Stage Goal

By the end of Stage 1, the team should have a working, documented, and reproducible traditional vulnerability-investigation baseline on the shared project target.
Minimum Stage 1 success consists of:

1. selecting and preparing the primary target;
2. establishing a conventional analysis and testing workflow;
3. completing at least one meaningful vulnerability investigation;
4. obtaining deterministic validation evidence where possible;
5. documenting the workflow and its manual bottlenecks so that later stages can reuse it.

The purpose is therefore not to demonstrate mastery of every traditional IoT vulnerability-discovery technique, but to establish a sufficiently realistic baseline for the remainder of the project.

3.2 Milestones

Stage 1 runs from Oct. 3 to Oct. 20, ending before Progress Update #1.
To avoid overcommitting to a detailed day-by-day schedule, Stage 1 will use three broad milestones.

Period	Milestone
Early Stage 1	Finalize the shared primary target, historical vulnerability task, workflow, and execution environment
Mid Stage 1	Execute and document the traditional vulnerability-investigation baseline
End of Stage 1	Validate results, summarize baseline measurements and manual bottlenecks, and prepare artifacts for Stage 2

By Progress Update #1, the team should be able to present the selected target, traditional tools and workflow, initial hands-on results, validation evidence, and candidate steps that may benefit from AI assistance.

3.3 Deliverables

Stage 1 will produce:

- a finalized primary target, version, component, and vulnerability task;
- a reproducible build and execution environment;
- a documented traditional vulnerability-investigation workflow;
- test inputs or harnesses used during validation;
- candidate and validated findings;
- runtime evidence such as crash or sanitizer output where applicable;
- baseline process measurements;
- notes on manual reasoning, setup difficulty, and workflow bottlenecks;
- scripts, commands, and configuration required to reproduce the experiment.

These artifacts will be handed off to Stage 2 for AI-assisted case studies and later reused where applicable by the Stage 3 agentic workflow.

3.4 Risks and Backup Plans

Risk	Backup Plan
Primary target is too large or difficult to build	Reduce the experiment to one component/parser or switch to the smaller backup target
Historical vulnerability is difficult to reproduce	Select another vulnerability in the same target or use a controlled known-vulnerable target
Runtime instrumentation is incompatible with the target	Use simpler execution-based validation or select a target compatible with ASan/debugging
Static analysis creates too many candidate findings	Narrow the analysis to the selected vulnerability type or component
Fuzzing requires excessive harness/setup work	Use manually constructed or existing test inputs; fuzzing is not required for minimum success
Firmware/hardware dependencies introduce excessive complexity	Run the software natively in Linux/container; embedded hardware is not required for the baseline
Stage 1 setup consumes too much of the schedule	Prioritize one validated investigation over additional targets or techniques

3.5 Resources and Logistics

The minimum Stage 1 environment is intentionally software-focused so that it can also be reused by the later agentic stage.

Required resources may include:

- Linux or Docker environment;
- C/C++ build toolchain;
- source-code search/browsing tools;
- AddressSanitizer or equivalent runtime instrumentation;
- optional debugger such as GDB;
- selected static-analysis or fuzzing tools if relevant to the final workflow;
- Git/version control;
- scripts and experiment logs.

Raspberry Pi, BeagleBone, cameras, smart-home devices, and other course hardware remain possible extension targets, but they are not required for minimum Stage 1 success.

Most required resources are open source or already available through existing course/team infrastructure, so the expected Stage 1 budget is minimal.

Stage 2 — Case Studies of AI-Assisted Vulnerability Discovery

1. Research and Related Works

Published work so far applies AI to almost every step of the traditional pipeline, but the evidence for the strongest

impact is in areas such as static analysis and fuzzing.

1.1 AI Applied to the Traditional Pipeline

Stage 1 defines the traditional workflow in seven steps: target understanding, attack-surface identification, static analysis, dynamic analysis and fuzzing, finding triage, root-cause analysis and vulnerability validation. We show how AI is applied at each step as well as what existing works we found within those areas through initial research.

Pipeline step	How AI is applied	Work found
1. Target understanding	Summarises code and flags trust boundaries from human-chosen context	Inside audits: Heelan, Big Sleep
2. Attack-surface identification	Finds or ranks the inputs worth testing	ChatHTTPFuzz: mutation points in IoT HTTP interfaces
3. Static analysis	Reviews code, infers taint sources and sinks, judges alerts	Source review: Heelan, Big Sleep. Taint and triage: IRIS, LLM4PFA, LATTE (firmware binaries)
4. Dynamic analysis and fuzzing	Drafts seeds, harnesses and protocol inputs; repairs failed emulation	Seeds and harnesses: BusyBox, OSS-Fuzz-Gen, PromptFuzz. Protocol input: ChatAFL, LLMIF, ChatHTTPFuzz. Rehosting repair: FirmPilot (new)
5. Finding triage	Groups duplicate crashes	GPTrace (LLM embeddings of crash reports)
6. Root-cause analysis	Explains a bug in a written report	Inside audits, such as Heelan's reports
7. Vulnerability validation	Generates proof inputs and patches	AIxCC systems, in a competition setting

Table summarising how AI is applied at each pipeline step. Highlighted stages present the most potential for AI.

1.2 Target Understanding and Attack Surfaces

Here the model is typically used to read code or documentation and summarise components, trust boundaries or entry points. In the works we found, humans choose to a large extent what the model gets to see. Setups range from giving the model specific code together with notes on the architecture and threat model [10], to letting an agent explore an entire codebase [11]. For IoT, some work has the model read protocol or interface data, such as HTTP requests, to pick out which fields or inputs are worth testing [12]. This suggests AI can help narrow down which code and inputs a human or downstream tool needs to examine, although how useful it is still depends on whether the human chose the right context.

1.3 Static Analysis

For static analysis, LLMs are able to perform a couple of tasks. They either review some provided code and reason about the data flow [10], [11], or augment the use of traditional static-analysis tools directly. In the second case the model can supply what the tool needs, such as which inputs and functions to treat as sources and sinks [13], [15], or it can review the tool's alerts afterwards and judge which are real and which are false positives [13], [14]. This suggests that AI can cut out a lot of the manual work needed in static analysis, however its outputs would still likely need verifying.

1.4 Dynamic Analysis

In fuzzing, the model typically prepares what the fuzzer needs to perform. Common uses are generating initial seed inputs [17], writing the harnesses that connect the fuzzer to the code [18], [19], and generating protocol-aware inputs, such as message templates or grammars, so that inputs are valid for network services [12], [20], [21]. However, AI use is prone to error and inaccuracies, leading to false positives and invalid or inefficient inputs [22]. AI has high potential in performing a lot of the manual preparation work for fuzzing processes.

1.5 Triage, Root Cause and Validation

After fuzzing or analysis, models can further be used to process findings. Typically this includes evaluating stack

traces and reports as well as explaining concepts and reasoning for bugs. This suggests AI can reduce the effort of sorting and explaining findings, although reliability concerns apply here as well.

1.6 Takeaways

Across each of the steps the usage of AI models follows a similar pattern. The model is either used to generate artifacts used in the analysis process and/or to analyse the results of existing tools and workflows. The steps with the clearest potential for AI augmentation are in static analysis and fuzzing. Therefore these are natural starting points for investigation. Other areas such as attack-surface identification, root-cause analysis, validation and firmware rehosting may also be good candidates to consider, however so far there is less evidence of AI application in these areas.
Following Stage 1's approach, we will prefer techniques that:

- report results we can compare with the Stage 1 baseline;
- work with human-provided context and a verifiable output;
- can run on the Stage 1 target with tools we can install;
- fit inside an agent loop as a single step or tool call, so Stage 3 can reuse them;
- are feasible within the two-week Stage 2 window.

2. Experimental Design

Stage 2 will study published AI-assisted techniques, reproduce results and processes where public artifacts allow, and apply selected techniques to the Stage 1 baseline, so we can see where and how much AI helps. Since the exact scope for the project has not been finalized, this section only gives some initial ideas that will be refined once Stage 1 is completed.

2.1 Objective

The objective is to procure and compare an AI-assisted version of the workflow outlined in Stage 1 with the Stage 1 baseline. The workflow should not be fully autonomous at this stage as a human will still be providing the context and checking every output. The work will show which AI use cases help, which do not, and which are worth carrying into Stage 3.

2.2 Approach

There are two initial approaches that we plan to use based on the results of Stage 1.
While Stage 1 builds the baseline, we will study selected papers and open-source systems until we understand how each of them work, we will then reproduce a reduced version of that result. For each, we record what task the AI performs, which traditional tools it works with, what context the model is given, how its output is checked, what results are reported, what limitations were observed, and whether the mechanism could be reused in Stage 3. We will ideally practice on known public vulnerable targets. This is to ensure that we understand how to apply each use case. This also allows us to easily adapt each use case to Stage 1's workflow and compare the AI-assisted result with the Stage 1 result.
Progress in developing the workflow in Stage 1 should also highlight to us areas where the AI could be applied to the traditional workflow. Where an area looks like it could use AI assistance, we can directly apply AI usage in that area whilst using literature as a reference for how others have done similar tasks.
Initial use cases to consider are:

- static-analysis triage and data-flow reasoning;
- fuzzing seed and harness generation;
- crash interpretation and root-cause hypotheses;

The first approach can start before Stage 1 finishes. The second depends on Stage 1 producing clear opportunities.
For direct comparisons, Stage 2 will reuse the Stage 1 baseline protocol: the same target where appropriate, the same information boundary, environment, and oracle. Prompt text and model context count as part of the information boundary and will be logged. For practice we may use small public targets that fit each use case, such as IoTGoat [26] or the target of a paper we reproduce.

2.3 Variables and Metrics

Stage 2 adopts the Stage 1 core metrics and controlled protocol so that AI-assisted results can be interpreted against the baseline. It additionally records:

- model and version, prompt and context supplied, and the number of repeated runs, because model output varies between runs;
- use-case-specific artifact quality, such as verdict agreement with validated results, harness build success, seed/input validity, or triage quality;
- API cost, run-to-run variation, human intervention, and observed failure modes.

The main variable is whether AI assists a given step, and with which use case. With few runs on one or two targets, results will be indicative rather than statistical. Well-known targets may overstate model performance if they appeared in training data.

3. Project Planning

We plan on running Stage 2 from Oct 21 to Nov 3, between Progress Update #1 and Progress Update #2. It will end by handing Stage 3 a selected set of AI techniques and design ideas.

3.1 Stage Goals

This stage is mainly based on studying existing papers and open-source systems, and reproducing them where we can. For each one, the goal is to understand:

- what task the AI system performs;
- which traditional tools it works with;
- what input and context is provided to the model;
- how the AI-generated output is validated;
- what results were reported;
- what limitations or failure modes were observed;
- which mechanisms are suitable for reuse or adaptation in Stage 3.

By Progress Update #2 on Nov 4, the team should have a clear summary of existing AI-assisted approaches, initial results on where AI helps compared with the Stage 1 baseline, and a selected set of techniques and architectural ideas for Stage 3.

3.2 Deliverables

Stage 2 will produce:

- a summary of existing AI-assisted approaches by pipeline area, with one filled template per source;
- notes on reproduced results, including what worked and what did not;
- initial observations from applying AI to the Stage 1 baseline, if the scope allows;
- a summary of failure modes and limitations, including cost and run-to-run variation;
- a shortlist of mechanisms and architectural ideas for Stage 3, with the context each needs and the tool it pairs with;
- the prompts, scripts and configuration needed to repeat what we ran.

3.3 Risks and Backup Plans

Risk	Backup plan
Stage 1's target or workflow is not ready by Oct 21	Run the practice and reproduction work only: reproduce results on public targets, and apply to the Stage 1 target later if time allows
Applying AI to the baseline takes longer than two weeks	Limit it to one or two use cases chosen for evidence and fit with the Stage 1 workflow
A reproduced result does not repeat, because of model changes or missing artifacts	Record the model and settings, report the difference, and treat the original result as unverified

Risk	Backup plan
Model output varies between runs	Repeat each run and report the spread instead of relying on one run
API cost or access limits	Use small prompts and few runs

3.4 Resources and Logistics

Papers, prompts and code are public. Hands-on work will need a Linux environment, the open-source tools Stage 1 selects and access to at least one LLM through some API service provider. We expect Stage 2 API usage to be about $5 and to stay under $10, using free and low-cost models and credits where possible, small prompts, and few repeated runs. Prompts, scripts and logs will be version controlled so Stage 3 can reuse them.

Stage 3 — Naptime-Inspired Agentic Vulnerability Discovery

Stage Positioning

Stage 3 builds on the first two stages:

- Stage 1 studies and executes the traditional vulnerability-discovery workflow;
- Stage 2 studies how AI can assist individual steps such as static analysis, fuzzing, harness generation, and triage;
- Stage 3 organizes these capabilities into an agentic vulnerability-research workflow that can autonomously invoke tools, observe results, and iteratively validate hypotheses.

For this stage, we plan to use Google Project Zero's Project Naptime as the primary methodological and architectural reference, and combine it with publicly available simplified implementations such as Baby Naptime to build a student-scale, Naptime-inspired reduced implementation.

The goal is not to reproduce Google's complete internal Naptime / Big Sleep system, nor to achieve production-scale vulnerability discovery. Instead, we aim to implement and evaluate a small agentic vulnerability-discovery system in a controlled environment, with emphasis on agent orchestration, tool use, runtime verification, human intervention, cost, and failure modes.

1. Research / Related Work

1.1 Problem Framing

Traditional LLM-based vulnerability detection often follows a one-shot pattern:

```
source code
    ↓
  LLM
    ↓
"potential vulnerability"
```

A limitation of this approach is that the model's conclusion may lack real execution evidence, and the model cannot naturally behave like a security researcher who repeatedly reads code, forms hypotheses, runs experiments, observes results, and revises its reasoning.

Therefore, the central question of Stage 3 is not:

“Can an LLM label vulnerable code?”

Instead, we ask:

Can an LLM-based agent reproduce part of a real vulnerability-research workflow by iteratively using source-analysis and runtime tools, and can its conclusions be verified by observable program behavior?

We use Project Naptime as the main methodological reference because it models vulnerability research as an iterative, hypothesis-driven, tool-grounded process:

```
Observe code / current state
            ↓
      Form hypothesis
            ↓
       Choose a tool
            ↓
```

```
Execute experiment
        ↓
Observe runtime evidence
        ↓
Update hypothesis
        ↓
Repeat until success or abandonment
```

In this architecture:

- the LLM performs reasoning and decides the next action;
- tools read source code, execute programs, and collect evidence;
- the controller / verifier determines whether a claimed finding is actually valid.

This is closer to a real security-research workflow than simply asking an LLM to classify code as vulnerable or non-vulnerable.

1.2 Core Related Work

1. Project Naptime — Google Project Zero

Project Naptime is the primary methodological reference for Stage 3.
Google publicly describes several core components:

- Code Browser: inspect functions, variables, and references;
- Python tool: perform calculations and generate complex test inputs;
- Debugger: execute the target and observe runtime state;
- Reporter / Controller: automatically verify whether a claimed success condition is actually satisfied.

The important contribution of Naptime is not only the use of a capable language model, but the combination of specialized tools + iterative agent loop + automatic verification, allowing the model to perform multi-step tasks that more closely resemble the workflow of a human security researcher.
Naptime is also directly relevant to our model–harness experiment. Project Zero reports that a certain level of model capability appears necessary to fully benefit from a complex task environment: the model must chain multiple tool calls, request source, generate reproduction code, run the target, and interpret the result. This motivates our controlled study of how a fixed harness interacts with different model capability levels.
Official resource: https://projectzero.google/2024/06/project-naptime.html

2. CyberSecEval 2 — Meta

CyberSecEval 2 provides public cybersecurity evaluation tasks, including software exploitation and memory-safety-related tasks.
Naptime uses these tasks to evaluate whether an agent can actually complete a security task rather than only generate plausible textual analysis.
The main lesson for our project is:
Stage 3 success conditions should rely on deterministic runtime evidence whenever possible.
Examples include:

- whether the target crashes;
- whether a sanitizer reports a memory error;
- whether a candidate input reliably reproduces the behavior;
- whether a known vulnerable condition is actually triggered.

Official research page: https://ai.meta.com/research/publications/cyberseceval-2-a-wide-ranging-cybersecurity-evaluation-suite-for-large-language-models/

3. From Naptime to Big Sleep — Google Project Zero + Google DeepMind

Naptime later evolved into Big Sleep and moved from benchmark tasks toward real-world vulnerability research.
Public examples show Big Sleep identifying a previously unknown exploitable memory-safety vulnerability in SQLite.
This demonstrates that a Naptime-style hypothesis-driven workflow can scale from controlled benchmark tasks to real codebases.

Big Sleep itself is not fully open source, so we do not plan to reproduce it. Instead, it serves as evidence that the core methodology can be extended to real-world vulnerability research.
Official resource: https://projectzero.google/2024/10/from-naptime-to-big-sleep.html

4. Baby Naptime — Open-Source Educational Reimplementation

Baby Naptime is an independent open-source simplified project inspired by Google Project Naptime.
It publicly implements:

- agent orchestration;
- LLM interface;
- code browser;
- GDB-based debugger;
- script runner;
- reporter;
- context summarization;
- C/C++ target compilation.

Its importance is that it demonstrates:
External developers can build a runnable student-scale Naptime-like system using the public research materials.
We will use Baby Naptime as an implementation reference, but Stage 3 should not simply fork and rename it. We will design our own reduced architecture based on our experimental goals.
Repository: https://github.com/faizann24/baby-naptime

5. PentestGPT

PentestGPT is an LLM-driven automated penetration-testing framework presented at USENIX Security 2024.
Its focus differs from Naptime: PentestGPT primarily targets penetration testing against running systems rather than source-based vulnerability research.
Its relevance to Stage 3 is mainly architectural:

- decomposing complex tasks into multiple roles / modules;
- interpreting tool outputs with LLMs;
- planning next actions based on observations;
- managing context over longer-running tasks.

PentestGPT therefore serves as complementary related work for agent orchestration and context management.
Paper: https://www.usenix.org/conference/usenixsecurity24/presentation/deng

6. VulnHuntr

VulnHuntr is an open-source LLM-assisted vulnerability-discovery tool focused on:

- remote user input;
- source code;
- call-chain exploration;
- remotely exploitable vulnerabilities.

It is primarily static / source-driven, which provides a useful contrast to Naptime's stronger emphasis on runtime verification.
Its main relevance to Stage 3 is:

- how an agent incrementally requests additional source context;
- how attacker-controlled input can be traced along call chains;
- how to avoid placing an entire codebase into the model context at once.

Repository: https://github.com/protectai/vulnhuntr

1.3 Related-Work Summary

The related work can be grouped into three relevant patterns:

Pattern	Representative Work	Main Evidence
Source-driven LLM analysis	VulnHuntr	source / call chain / static reasoning
Tool-using vulnerability research agent	Naptime / Big Sleep	source + runtime + debugger / execution
Agentic penetration testing	PentestGPT	running service / network behavior / tool output

Stage 3 will primarily adopt the second pattern:
source understanding + iterative tool use + runtime verification
At the same time, we may borrow:

- source-context / call-chain exploration ideas from VulnHuntr;
- orchestration / context-management ideas from PentestGPT;
- practical implementation structure from Baby Naptime.

2. Experimental Design

2.1 Research Questions

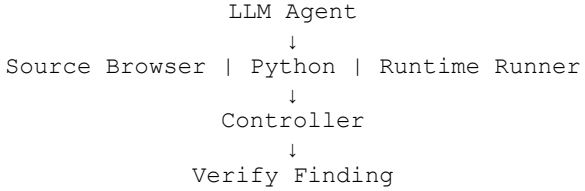
Stage 3 initially focuses on the following questions:
RQ1 — Feasibility
Can a student-scale Naptime-inspired agent complete an end-to-end vulnerability investigation on controlled C/C++ targets?
RQ2 — Value of Runtime Tools
Does giving the agent runtime verification capability improve the reliability of its findings compared with source-only reasoning?
RQ3 — IoT / Embedded-Linux Adaptation
Can the same agentic workflow be applied to controlled IoT / embedded-Linux-relevant software targets?
RQ4 — Model–Harness Interaction
With the vulnerability-research harness held fixed, how does underlying model capability affect the benefit obtained from the harness?
This question evaluates two competing effects:

- Capability Compensation: weaker models may gain more because the harness externalizes capabilities such as search, execution, and verification.
- Capability Amplification: stronger models may gain more because they are better able to plan, invoke, and interpret multi-step tool use.

A third possibility is a threshold + saturation pattern: very weak models may not reliably use the harness, medium-capability models may obtain the largest relative gain, and the strongest models may achieve the highest absolute performance but smaller relative gains due to ceiling effects.
Optional RQ5 — Context / Protocol Awareness
If the target has a network or IoT protocol, does providing protocol/context information help the agent generate more meaningful inputs or reach relevant code paths more efficiently?
RQ5 is a stretch question rather than a core success condition.

2.2 Proposed Reduced Architecture

We plan to implement a reduced Naptime-like architecture:

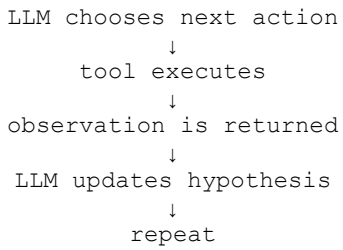


Minimum Tool Set

The first version only requires:

```
read_source()
find_references()
run_python()
run_target(input)
```
Optional extensions:
```
run_gdb()
run_semgrep()
collect_coverage()
```

Core Agent Loop



The controller will enforce limits such as:

- maximum number of steps;
- runtime timeout;
- allowed tools;
- allowed file / process scope.

2.3 Candidate Target Criteria

Stage 3 will use the Stage 1 primary target for the main cross-stage comparison under the same information boundary and deterministic oracle. Additional controlled targets may be selected for MVP development and as a generalization check rather than as substitutes for the baseline comparison.

An ideal target should satisfy as many of the following as possible:

- open source;
- C / C++;
- compilable and runnable on Linux / in a container;
- compatible with ASan / debugger instrumentation;
- small enough for an agent to understand;
- documented / historical vulnerability or other ground truth;
- clear external input;
- relevant to embedded Linux / IoT usage;
- preferably exposes a protocol / network interface to strengthen IoT relevance.

Initial plan:

- the Stage 1 primary target for the main baseline-to-agent comparison;
- 1 simple controlled/benchmark target for MVP development and, if time permits, 1 additional IoT/embedded-Linux-relevant target as a generalization check.

Any additional targets will be finalized before the corresponding Stage 3 experiment. For the shared target, the Stage 1 repository/revision, task information boundary, environment, and oracle remain fixed unless a documented technical constraint requires a deviation.

2.4 Experimental Conditions

To separate the effects of different capabilities in the agent architecture, we plan to compare at least the following conditions.

Condition A — Source-Only Agent

The agent can:

- read source code;
- find references;
- reason about possible vulnerabilities.

It cannot execute the target.
Its output is therefore only a candidate finding.

Condition B — Source + Runtime Verification

The agent can additionally:

- generate candidate input;
- execute the target;
- collect stdout / stderr;
- inspect ASan / crash evidence.

This will be the main Stage 3 system configuration.

Optional Condition C — Source + Runtime + Additional Context

If a target exposes a protocol or a clear interface specification, the agent may also receive:

- protocol documentation;
- valid message grammar;
- example inputs;
- component / interface description.

We can then evaluate whether additional context improves:

- input quality;
- relevant code-path reachability;
- number of tool calls;
- time / token cost.

2.5 Model–Harness Interaction Experiment

After the core harness is stable, we will run a secondary controlled experiment that holds the target and harness configuration constant while varying the underlying model.
The purpose is to measure whether the same agent harness benefits weaker and stronger models differently.

Competing Hypotheses

- H1 — Capability Compensation: weaker models gain more because the harness externalizes capabilities they lack.
- H2 — Capability Amplification: stronger models gain more because they are better able to plan, use tools, interpret observations, and maintain long-horizon trajectories.
- H3 — Threshold + Saturation: very weak models cannot reliably use the harness; medium-capability models obtain the largest relative gain; strong models achieve the highest absolute performance but may experience diminishing relative gains.

Model Levels

We will select at least two, and preferably three, model capability / cost tiers:

- Weak / low-cost model
- Medium model
- Strong model

Exact models will be selected before the formal experiment. Where possible, they will use the same task prompt, target, tool interface, step limit, verifier, and comparable sampling settings.

Harness Levels

If experiment time permits, we will compare multiple harness levels:

- H0 — Minimal / bare model
- H1 — Basic harness: source browsing + target execution
- H2 — Full reduced Naptime harness: source browsing + Python + runtime verification + controller

This yields a model × harness matrix:

Model Capability	H0 Minimal	H1 Basic Harness	H2 Full Harness
Weak			
Medium			
Strong			

The key quantity is not only final model performance, but the harness gain for each model:

```
Harness Gain(model) = Performance(full harness) - Performance(minimal harness)
```

This allows us to study whether model capability and harness capability compensate for, amplify, or constrain each other.

2.6 Experimental Procedure

For each measured condition, the basic procedure will be:

1. Prepare known ground truth for evaluation and freeze the task information boundary.
2. Build the target in the controlled runtime environment; for the primary comparison, reuse the Stage 1 environment and oracle.
3. Use a fixed agent configuration, allowed tool set, step limit, and verifier for the condition.
4. Start the vulnerability-investigation task.
5. Record the complete trajectory.
6. If the agent produces a candidate finding, have the controller verify it by actually executing the relevant test.
7. Repeat each important condition a fixed number of times to estimate success rate and run-to-run variation.
8. Compare results across experimental conditions.

For known-vulnerability targets:

- the agent receives only the information permitted by the frozen boundary and will not be given the final vulnerable function/location, PoC, or solution;
- ground truth will be used only for experimental evaluation and deterministic validation.

2.7 Metrics

Primary Metrics

- Validated finding / success rate: whether the system triggers and validates known vulnerable behavior, reported over a fixed number of repeated runs per condition;
- False candidate finding: a claimed vulnerability that fails the deterministic oracle;
- Steps / tool calls: number of agent actions required to complete the investigation, treated as an agent-specific process measure rather than as equivalent to human steps;
- Human intervention: number and type of manual interventions required;
- Harness gain: improvement from minimal/no harness to the fixed agent harness for each model tier, using the same task and verifier.

Secondary Metrics

- execution time, reported descriptively rather than as a claim that the agent is faster than human analysts or Naptime/Big Sleep;
- token / API cost;
- cost per validated successful finding;

- tool-call error rate;
- reproducibility and run-to-run variance across repeated runs;
- oracle evidence, such as runtime crash / sanitizer evidence;
- context size;
- failure-mode / bottleneck category.

Qualitative Analysis

We will also record representative trajectories such as:
- agent forms a correct hypothesis;
- agent revises an incorrect hypothesis after observing runtime evidence;
- agent becomes stuck in irrelevant code;
- agent generates invalid input;
- model hallucination;
- tool / environment failure.

3. Project Planning

3.1 Stage 3 Goals

The core Stage 3 goals are:
1. Understand and reconstruct a Naptime-style agent architecture.
2. Implement a minimum viable agent loop.
3. Integrate source browsing and at least one runtime-verification capability.
4. Complete an end-to-end investigation on a controlled target.
5. Run experiments on 1–2 IoT / embedded-Linux-relevant targets.
6. Record and analyze effectiveness, cost, human intervention, reproducibility, and failure modes.

3.2 Timeline and Milestones

Nov. 4 – Nov. 17: Stage 3A — Architecture and MVP

Main tasks:
- read official Naptime materials;
- study the Baby Naptime codebase;
- define the reduced architecture;
- implement the agent loop;
- implement source-browser tooling;
- implement runtime runner / ASan verification;
- complete an end-to-end run on one simple C/C++ target.

Nov. 18: Progress Update #3

Expected demonstration:
- reference architecture;
- our reduced design;
- implemented tool interfaces;
- at least one end-to-end MVP demonstration;
- current blockers / risks;
- Stage 3B target selection and experimental plan.

Nov. 18 – Dec. 3: Stage 3B — Integration, Experiments, and Evaluation

Main tasks:

- finalize and prepare 1–2 primary targets;
- complete agent / tool integration;
- run controlled experiments;
- run the model–harness interaction experiment after the core harness is stable;
- collect metrics;
- repeat important experiments;
- analyze failure modes;
- prepare the final demo and results.

Dec. 4: Final Presentation

Expected content:

- Stage 1 → Stage 2 → Stage 3 progression;
- Stage 3 architecture;
- working agent demo / recorded trajectory;
- experimental results;
- runtime-validated findings;
- limitations / failure modes;
- future work.

Dec. 4 – Dec. 9: Final Report

Main tasks:

- consolidate experiments;
- complete any necessary validation;
- finalize result analysis;
- document methodology, limitations, risks, and lessons learned.

3.3 Deliverables

Stage 3 is expected to produce:

D1 — Reduced Naptime-Inspired Agent

Including:

- agent loop;
- model interface;
- source-browsing tools;
- runtime execution / verification tool;
- controller;
- structured logging.

D2 — Reproducible Target Environment

Including:

- build instructions;
- container / environment setup;
- target versions;
- ground truth;
- experiment configuration.

D3 — Experimental Dataset / Logs

Recording:

- trajectories;
- tool calls;
- model outputs;
- runtime results;
- timing;
- cost;
- success / failure;
- intervention;
- failure modes.

D4 — Evaluation and Case Studies

Summarizing:

- successful investigations;
- failed investigations;
- source-only vs runtime-assisted behavior;
- model–harness interaction and harness gain across model tiers;
- IoT / embedded-Linux adaptation;
- limitations and lessons learned.

3.4 Risks and Backup Plans

Risk 1 — Naptime implementation details are not fully public

Risk: Google has not released the complete Naptime / Big Sleep source code.
Backup: Treat Naptime as a methodological reference rather than a codebase to reproduce. Use Baby Naptime and other open-source systems as implementation references.

Risk 2 — Agent infrastructure takes too long to build

Risk: Tool orchestration, debugger integration, context management, or error handling may consume too much time.
Backup: Keep the minimum tool set small:

```
read_source()
find_references()
run_python()
run_target()
```

Use ASan-based execution before adding full GDB integration.

Risk 3 — Selected real-world target is too large or difficult

Risk: The agent may not handle a large codebase or complicated build environment.
Backup: Use a smaller known-vulnerability target or isolate one vulnerable component / parser.

Risk 4 — Real IoT hardware introduces excessive setup complexity

Risk: Course hardware or commercial devices may require cross-compilation, firmware extraction, special networking, or unavailable debugging access.
Backup: Complete the core experiments on IoT-relevant open-source software in normal Linux / containers. Raspberry Pi / embedded hardware is an extension, not a prerequisite.

Risk 5 — Agent does not discover new vulnerabilities

Risk: Discovering a new zero-day is not a realistic course requirement.
Backup: Evaluate on historically vulnerable or controlled targets with known ground truth. Success means rediscovering / validating known vulnerable behavior, not discovering an unknown vulnerability.

Risk 6 — LLM results are nondeterministic

Risk: Repeated runs may produce different trajectories.
Backup: Fix the model / version / settings where possible, log complete trajectories, repeat important experiments, and report variance rather than relying on a single run.

Risk 7 — Training-data contamination

Risk: A well-known historical vulnerability or benchmark may have appeared in model training data, making rediscovery easier than a genuinely unseen task.
Backup: Treat results as controlled-case evidence, avoid claims of general vulnerability-discovery capability, and use an additional target or vulnerability as a generalization check where feasible.

Risk 8 — Information-boundary leakage

Risk: Prompts, documentation, filenames, comments, or prior analyst knowledge may unintentionally reveal the vulnerable location or solution.
Backup: Freeze and review the information boundary before measured runs, log all model context, and document any leakage or protocol deviation.

Risk 9 — Setup time dominates the comparison

Risk: Build repair, dependency installation, harness preparation, or instrumentation may dominate wall-clock time and obscure the vulnerability-analysis process.
Backup: Record setup time separately from investigation time and interpret time descriptively rather than as a direct human-versus-agent performance ranking.

3.5 Resources and Budget

Required Software / Infrastructure

- Linux environment or Docker;
- C/C++ compiler;
- AddressSanitizer;
- optional GDB;
- Python;
- source-search / indexing tools;
- selected LLM API or local-model interface;
- Git / GitHub;
- controlled open-source targets.

Optional Hardware

- Raspberry Pi / other course embedded-Linux hardware;
- isolated networking environment.

Hardware is not required for the minimum viable project.

Expected Budget

The primary potential cost is:

- LLM API usage.

The project should otherwise rely mainly on:

- open-source software;
- existing team laptops / school compute;
- course-provided hardware.

API cost will be recorded as one of the experimental metrics and controlled using:

- maximum step limits;
- restricted context;
- small target scope;
- capped repeated trials.

We expect Stage 3 API usage to be about $10 and to stay under $40, relying mainly on free credits and low-cost model tiers, with the strongest tier used for only a limited number of trials.

4. Stage 3 Success Criteria

Minimum Success

- reduced agent architecture implemented;
- source browsing + runtime execution works;
- agent completes one controlled end-to-end vulnerability investigation;
- the result is verified using deterministic runtime evidence.

Core Success

- experiments completed on 1–2 controlled IoT / embedded-Linux-relevant targets;
- repeated trials and metrics collected;
- meaningful comparison between source-only and runtime-assisted behavior;
- a model–harness comparison completed with at least two model capability tiers;
- failure modes documented.

Stretch Success

- protocol/context-aware comparison;
- GDB or additional static-analysis integration;
- execution on Raspberry Pi / course hardware;
- larger real-world target;
- more autonomous planning / reduced human intervention.

Shared Experimental Limitations

Analyst experience. The Stage 1 analysts are beginners. Human time-to-finding therefore reflects learning, environment familiarity, and security-analysis experience as well as the workflow itself. Time is reported as descriptive context, with setup and investigation separated where possible, and is not used to claim that the agent is faster than expert humans, Naptime, or Big Sleep.

Scale and statistics. The project uses a small number of targets and one or two known vulnerabilities per target. Results are therefore case-study evidence rather than statistically general claims. We emphasize validated success, false candidate findings, reproducibility, intervention, cost, and qualitative failure modes rather than precision/recall/F1 unless enough independent ground-truth cases are available.

Known-target and training-data effects. Historical vulnerabilities are necessary to provide ground truth, but popular targets or CVEs may have appeared in model training data. This can overstate AI performance. Additional targets are used as generalization checks where feasible, and the limitation will be stated explicitly.

Information-boundary sensitivity. Task difficulty can change substantially depending on whether a condition receives a CVE identifier, CWE type, component name, protocol documentation, example inputs, or other hints. The boundary is therefore frozen and logged for each comparison, and any deviation is treated as a limitation rather than silently reconciled.

Metric comparability. Human actions and agent tool calls are useful for describing their respective workflows but are not equivalent units of effort. Likewise, wall-clock efficiency can be dominated by setup and tool maturity. Cross-stage interpretation therefore focuses on the pattern of validated outcomes and failure modes under the shared protocol rather than on a single aggregate score.

Conclusion

This document sets out a three-stage plan for studying how AI changes vulnerability discovery in IoT and embedded-Linux software. Stage 1 establishes a documented human-led baseline protocol using traditional tools. Stage 2 studies human-guided AI assistance within that protocol, while accounting for analyst carryover and measuring selected AI artifacts where a clean end-to-end rerun is not possible. Stage 3 combines the most promising capabilities into a reduced, Naptime-inspired agent and evaluates it on the shared baseline target, with additional targets used as generalization checks.

The central comparison is made under a shared target, information boundary, environment, and deterministic oracle. Under those controls, we ask how traditional, AI-assisted, and agentic analysis differ in validated success, false candidate findings, reproducibility, human intervention, cost, and failure modes. Findings count only when they pass the oracle, and the artifacts produced by each stage - environments, scripts, prompts, logs, and bottleneck notes - are handed to the next stage so work can be reused and protocol deviations can be tracked.

The expected outcome is an evidence-based case study of where AI assistance helps, where it does not, and what it costs in reliability, intervention, and API usage, together with a working student-scale agent and a catalogue of failure modes. The project is deliberately scoped for a short course schedule, so conclusions will remain indicative rather than statistical and will be interpreted alongside the shared experimental limitations above. Success is defined as rediscovering and validating known vulnerable behavior, not as finding new zero-days.

Extensions such as protocol-aware context, debugger and additional static-analysis integration, larger real-world targets, and execution on Raspberry Pi or other course hardware remain possible future work if time allows.

References

[1] J. Chen, W. Diao, Q. Zhao, C. Zuo, Z. Lin, X. Wang, W. C. Lau, M. Sun, R. Yang, and K. Zhang, “IoTFuzzer: Discovering Memory Corruptions in IoT Through App-based Fuzzing,” in Network and Distributed System Security Symposium (NDSS), 2018.
[2] Y. Zheng, A. Davanian, H. Yin, C. Song, H. Zhu, and L. Sun, “FIRM-AFL: High-Throughput Greybox Fuzzing of IoT Firmware via Augmented Process Emulation,” in 28th USENIX Security Symposium, pp. 1099–1114, 2019.
[3] W. Xie, J. Chen, Z. Wang, C. Feng, E. Wang, Y. Gao, B. Wang, and K. Lu, “Game of Hide-and-Seek: Exposing Hidden Interfaces in Embedded Web Applications of IoT Devices,” in The ACM Web Conference (WWW), pp. 524–532, 2022.
[4] X. Feng, R. Sun, X. Zhu, M. Xue, S. Wen, D. Liu, S. Nepal, and Y. Xiang, “Snipuzz: Black-box Fuzzing of IoT Firmware via Message Snippet Inference,” in ACM SIGSAC Conference on Computer and Communications Security (CCS), pp. 337–350, 2021.
[5] W. Gibbs et al., “Operation Mango: Scalable Discovery of Taint-Style Vulnerabilities in Binary Firmware Services,” in 33rd USENIX Security Symposium, 2024.
[6] D. D. Chen, M. Egele, M. Woo, and D. Brumley, “Towards Automated Dynamic Analysis for Linux-based Embedded Firmware,” in Network and Distributed System Security Symposium (NDSS), 2016.
[7] M. Kim, D. Kim, E. Kim, S. Kim, Y. Jang, and Y. Kim, “FirmAE: Towards Large-Scale Emulation of IoT Firmware for Dynamic Analysis,” in Annual Computer Security Applications Conference (ACSAC), pp. 733–745, 2020.
[8] N. Redini, A. Machiry, R. Wang, C. Spensky, A. Continella, Y. Shoshitaishvili, C. Kruegel, and G. Vigna, “KARONTE: Detecting Insecure Multi-binary Interactions in Embedded Firmware,” in IEEE Symposium on Security and Privacy, pp. 1544–1561, 2020.
[9] OWASP, "Firmware Security Testing Methodology (FSTM)," GitHub repository. [Online]. Available: https://github.com/scriptingxss/owasp-fstm (accessed Oct. 2, 2026).
[10] S. Heelan, "How I used o3 to find CVE-2025-37899, a remote zeroday vulnerability in the Linux kernel's SMB implementation," Sean Heelan's Blog, May 22, 2025. [Online]. Available: https://sean.heelan.io/2025/05/22/how-i-used-o3-to-find-cve-2025-37899-a-remote-zeroday-vulnerability-in-the-linux-kernels-smb-implementation/ (accessed Oct. 2, 2026).
[11] Big Sleep team, Google Project Zero and Google DeepMind, "From Naptime to Big Sleep: Using large language models to catch vulnerabilities in real-world code," Project Zero Blog, Nov. 1, 2024. [Online]. Available: https://projectzero.google/2024/10/from-naptime-to-big-sleep.html (accessed Oct. 2, 2026).
[12] Z. Yang, H. Peng, Y. Jiang, X. Li, H. Du, S. Wang, and J. Liu, "ChatHTTPFuzz: Large language model-assisted IoT HTTP fuzzing," arXiv:2411.11929, 2024.
[13] Z. Li, S. Dutta, and M. Naik, "IRIS: LLM-assisted static analysis for detecting security vulnerabilities," in Proc. Int. Conf. Learn. Represent. (ICLR), 2025.
[14] X. Du, K. Yu, C. Wang, Y. Zou, W. Deng, Z. Ou, X. Peng, L. Zhang, and Y. Lou, "Minimizing false positives in static bug detection via LLM-enhanced path feasibility analysis," arXiv:2506.10322, 2025.
[15] P. Liu, C. Sun, Y. Zheng, X. Feng, C. Qin, Y. Wang, Z. Xu, Z. Li, P. Di, Y. Jiang, and L. Sun, "LLM-powered static binary taint analysis," ACM Trans. Softw. Eng. Methodol., Mar. 2025, doi: 10.1145/3711816 (arXiv:2310.08275).
[16] S. Ullah, M. Han, S. Pujar, H. Pearce, A. Coskun, and G. Stringhini, "LLMs cannot reliably identify and reason about security vulnerabilities (yet?): A comprehensive evaluation, framework, and benchmarks," in Proc. IEEE Symp. Secur. Privacy (SP), 2024.
[17] Asmita, Y. Oliinyk, M. Scott, R. Tsang, C. Fang, and H. Homayoun, "Fuzzing BusyBox: Leveraging LLM and crash reuse for embedded bug unearthing," in Proc. 33rd USENIX Secur. Symp., Philadelphia, PA, USA, Aug. 2024, pp. 883-900.
[18] O. Chang, D. Liu, and J. Metzman, "Leveling up fuzzing: Finding more vulnerabilities with AI," Google Security Blog, Nov. 20, 2024. [Online]. Available: https://security.googleblog.com/2024/11/leveling-up-fuzzing-finding-more.html (accessed Oct. 2, 2026).
[19] Y. Lyu, Y. Xie, P. Chen, and H. Chen, "Prompt fuzzing for fuzz driver generation," in Proc. ACM SIGSAC Conf. Comput. Commun. Secur. (CCS), Salt Lake City, UT, USA, Oct. 2024, pp. 3793-3807.
[20] R. Meng, M. Mirchev, M. Böhme, and A. Roychoudhury, "Large language model guided protocol fuzzing," in Proc. 31st Annu. Netw. Distrib. Syst. Secur. Symp. (NDSS), 2024.
[21] J. Wang, L. Yu, and X. Luo, "LLMIF: Augmented large language model for fuzzing IoT devices," in Proc. IEEE Symp. Secur. Privacy (SP), 2024, pp. 881-896, doi: 10.1109/SP54263.2024.00211.
[22] Y. Jiang et al., "When fuzzing meets LLMs: Challenges and opportunities," arXiv:2404.16297, Apr. 2024 (FSE '24, Porto de Galinhas, Brazil).
[23] Y. Shen, F. Zhang, and H. Xu, "FirmPilot: Evidence-guided multi-agent environment recovery for IoT firmware rehosting," arXiv:2607.14903, 2026.
[24] P. Herter, V. Ahlrichs, R. Açilan, and J. Horsch, "GPTrace: Effective crash deduplication using LLM embeddings," in Proc. IEEE/ACM Int. Conf. Softw. Eng. (ICSE), Rio de Janeiro, Brazil, Apr. 2026.
[25] DARPA, "AI Cyber Challenge marks pivotal inflection point for cyber defense," DARPA News, Aug. 8, 2025. [Online]. Available: https://www.darpa.mil/news/2025/aixcc-results (accessed Oct. 2, 2026).
[26] OWASP, "IoTGoat," GitHub repository. [Online]. Available: https://github.com/OWASP/IoTGoat (accessed Oct. 2, 2026).
[27] OWASP, "IoT Security Testing Guide (ISTG)," GitHub repository. [Online]. Available: https://github.com/OWASP/owasp-istg (accessed Oct. 2, 2026).
[28] OWASP, "IoT Security Verification Standard (ISVS)," GitHub repository. [Online]. Available: https://github.com/OWASP/IoT-Security-Verification-Standard-ISVS (accessed Oct. 2, 2026).
[29] F. Chantzis, I. Stais, P. Calderon, E. Deirmentzoglou, and B. Woods, Practical IoT Hacking: The Definitive Guide to Attacking the Internet of Things. San Francisco, CA, USA: No Starch Press, 2021.
[30] A. Zeller, R. Gopinath, M. Böhme, G. Fraser, and C. Holler, The Fuzzing Book. Saarbrücken, Germany: CISPA and Saarland University, 2019. [Online]. Available: https://www.fuzzingbook.org.
[31] H. Liang, X. Pei, X. Jia, W. Shen, and J. Zhang, "Fuzzing: State of the art," IEEE Trans. Rel., vol. 67, no. 3, pp. 1199-1218, 2018, doi: 10.1109/TR.2018.2834476.
[32] A. Fioraldi, D. Maier, H. Eißfeldt, and M. Heuse, "AFL++: Combining incremental steps of fuzzing research," in Proc. 14th USENIX Workshop Offensive Technol. (WOOT), Aug. 2020. [Online]. Available: https://www.usenix.org/conference/woot20/presentation/fioraldi.
[33] G. T. Klees, A. Ruef, B. Cooper, S. Wei, and M. Hicks, "Evaluating fuzz testing," in Proc. ACM SIGSAC Conf. Comput. Commun. Secur. (CCS), Toronto, ON, Canada, Oct. 2018.

[34] M. Schloegel et al., "SoK: Prudent evaluation practices for fuzzing," in Proc. IEEE Symp. Secur. Privacy (SP), San Francisco, CA, USA, May 2024, doi: 10.1109/SP54263.2024.00137.
[35] Y. Deng, C. S. Xia, H. Peng, C. Yang, and L. Zhang, "Large language models are zero-shot fuzzers: Fuzzing deep-learning libraries via large language models," in Proc. 32nd ACM SIGSOFT Int. Symp. Softw. Testing Anal. (ISSTA), 2023, pp. 423-435.
[36] C. S. Xia, M. Paltenghi, J. Le Tian, M. Pradel, and L. Zhang, "Fuzz4All: Universal fuzzing with large language models," in Proc. IEEE/ACM 46th Int. Conf. Softw. Eng. (ICSE), 2024.
[37] C. Zhang, Y. Zheng, M. Bai, Y. Li, W. Ma, X. Xie, Y. Li, L. Sun, and Y. Liu, "How effective are they? Exploring large language model based fuzz driver generation," in Proc. 33rd ACM SIGSOFT Int. Symp. Softw. Testing Anal. (ISSTA), Vienna, Austria, Sep. 2024.
[38] X. Ma, L. Luo, and Q. Zeng, "From one thousand pages of specification to unveiling hidden bugs: Large language model assisted fuzzing of Matter IoT devices," in Proc. USENIX Secur. Symp., 2024, pp. 4783-4800.
[39] H. Liu et al., "Labrador: Response guided directed fuzzing for blackbox IoT devices," in Proc. IEEE Symp. Secur. Privacy (SP), 2024, pp. 1920-1938.
[40] J. Ji, C. Zhang, S. Gan, L. Jian, H. Liu, T. Liu, L. Zheng, and Z. Jia, "FirmAgent: Leveraging fuzzing to assist LLM agents with IoT firmware vulnerability discovery," in Proc. Netw. Distrib. Syst. Secur. Symp. (NDSS), San Diego, CA, USA, Feb. 2026, doi: 10.14722/ndss.2026.231943.
[41] Z. Jia et al., "PANGOLIN: Fuzzing multilingual IoT firmware with LLM-driven code analysis," in Proc. 35th USENIX Secur. Symp., Baltimore, MD, USA, Aug. 2026, pp. 673-692.
[42] S. Glazunov and M. Brand, "Project Naptime: Evaluating offensive security capabilities of large language models," Google Project Zero Blog, Jun. 20, 2024. [Online]. Available: https://projectzero.google/2024/06/project-naptime.html.
[43] Semgrep, "semgrep," GitHub repository. [Online]. Available: https://github.com/semgrep/semgrep (accessed Oct. 2, 2026).

[44] GitHub, "CodeQL." [Online]. Available: https://codeql.github.com.